Decomposing the Training of Deep Learned Turbo codes via a Feasible MAP Decoder [1]

Abhijeet Mulgund

University of Illinois Chicago

September 5, 2023





[1]This work was supported by NSF under awards 1934915 and 2217023, and the NRDI Office within the framework of the Artificial Intelligence National Laboratory program (MILAB), Hungary.



Natasha Devroye



György Turán



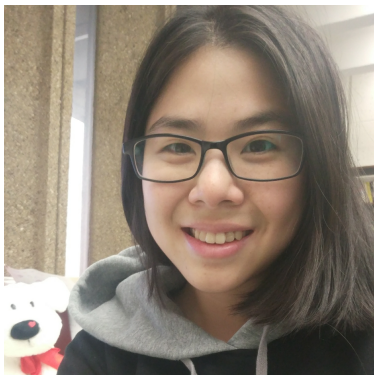
Miloš Žefran



Abhijeet Mulgund



Raj Shekhar



Yeqi Wei



Yingyao Zhou

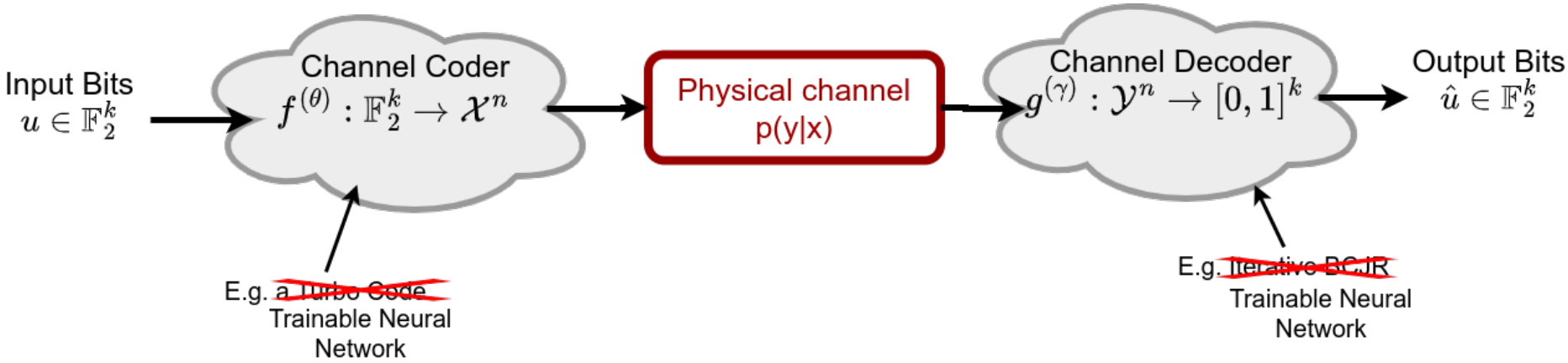


Vincent Dumoulin

Outline

1. What are Deep-Learned Error Correcting Codes (DL-ECCs)?
2. Decomposition of Binary Cross-Entropy
3. Decomposition Estimation via the Junction-Tree Algorithm

What are DL-ECCs?



Trained by minimizing

$$\frac{1}{k}\sum_{j=1}^{k}\mathbb{E}[\text{BCE}(\mathbf{u}_j, g_j^{(\gamma)}(\mathbf{y}))] = -\frac{1}{k}\sum_{j=1}^{k}\mathbb{E}[\mathbb{1}_{\mathbf{u}_j=1}\log g_j^{(\gamma)}(\mathbf{y}) + \mathbb{1}_{\mathbf{u}_j=0}\log(1-g_j^{(\gamma)}(\mathbf{y}))]$$

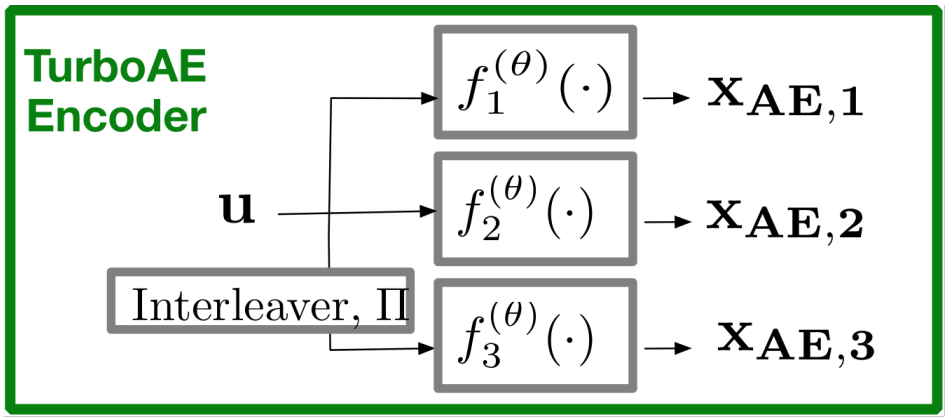
- TurboAE: *Jiang et al.* '19
- DeepCode: *Kim et al.* '20 and GBAF: *Ozfatura et al.* '22 - incorporates channel feedback
- Ko codes: *Makkuva et al.* '21

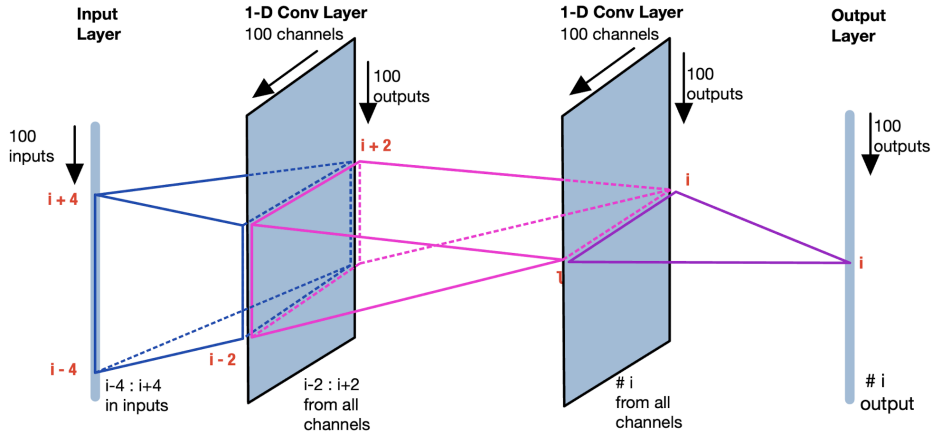
How can we interpret these black boxes?

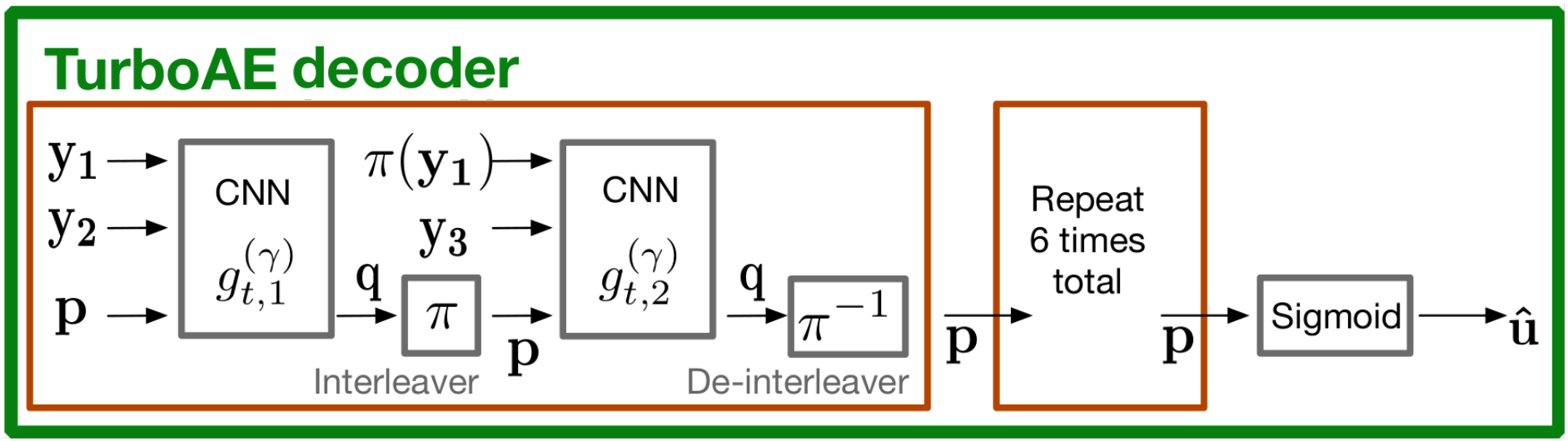
Value in Interpretations

1. Convert DL-ECCs to well-known codes with theory and efficient implementations - *Post-hoc Interpretation*
   - Exact Conversion
   - Approximate Conversion
2. Understand the training process to further improve DL-ECC and understand what Deep Learning brings to coding.
   - complementary to broader general deep learning questions of implicit regularization, DL learnability of problems, and loss landscapes.
3. DL-ECCs form a mathematically well-defined optimization problem to use as a sandbox for developing general purpose interpretability techniques.

TurboAE







[1] Figures adapted from *Jiang et al., NeurIPS '19* and *Mulgund et al., Allerton '22*

Understanding Cross-Entropy Minimization

We can decompose cross-entropy into a KL-Divergence term and an entropy term

$$\begin{aligned}\mathbb{C}_j(\theta,\gamma) &:= \mathbb{E}[\mathrm{BCE}(\mathbf{u}_j, g_j^{(\gamma)}(\mathbf{y}))] \\ &= D_{KL}(\mathbb{P}[\mathbf{u}_j|\mathbf{y}] \| g_j^{(\gamma)}(\mathbf{y})|\mathbf{y}) + \mathbb{H}(\mathbf{u}_j|\mathbf{y})\end{aligned}$$

where $\mathbf{y}$ depends only on θ and

$$D_{KL}(\mathbb{P}[\mathbf{u}_j|\mathbf{y}] \| g_j^{(\gamma)}(\mathbf{y})|\mathbf{y}) = \mathbb{E}\left[D_{KL}(\mathbb{P}[\mathbf{u}_j|\mathbf{y}] \| g_j^{(\gamma)}(\mathbf{y}))\right]$$

$$\mathbb{H}(\mathbf{u}_j|\mathbf{y}) = \mathbb{E}\left[\mathbb{H}_2(\mathbb{P}[\mathbf{u}_j|\mathbf{y}])\right]$$

Understanding Cross-Entropy Minimization

We can decompose cross-entropy into a KL-Divergence term and an entropy term

$$\begin{aligned}\mathbb{C}_j(\theta,\gamma) &:= \mathbb{E}[\mathrm{BCE}(\mathbf{u}_j, g_j^{(\gamma)}(\mathbf{y}))] \\ &= D_{KL}(\mathbb{P}[\mathbf{u}_j|\mathbf{y}] \| g_j^{(\gamma)}(\mathbf{y})\big|\mathbf{y}) + \mathbb{H}(\mathbf{u}_j|\mathbf{y})\end{aligned}$$

- $\mathbb{H}(\mathbf{u}_j|\mathbf{y})$ depends only on the **encoder**!
- For a fixed encoder, $D_{KL}(\mathbb{P}[\mathbf{u}_j|\mathbf{y}] \| g_j^{(\gamma)}(\mathbf{y})\big|\mathbf{y})$ is minimized to 0 when $g_j^{(\gamma)} = \mathbb{P}[\mathbf{u}_j|\mathbf{y} = \cdot]$, i.e. when $g^{(\gamma)}$ is the **MAP decoder**.

We can understand cross-entropy minimization as a back-and-forth between

1. Finding an encoder
2. Finding a suitable decoder for the encoder

TurboAE Training

- Training is done back-and-forth: $g^{(\gamma)}$ is trained for 125 batches, then $f^{(\theta)}$ is trained for 25 batches. This is one epoch.
- Trained for 800 epochs.
- Channel SNR (SNR$= -10\log_{10}\sigma^2$) is fixed at 2.0 for encoder, and is sampled between -1.5-2.0 for the decoder.

Can we use the BCE decomposition to study how the encoder and decoder are optimized?

If we could compute $\mathbb{P}[\mathbf{u}_j|\mathbf{y}]$ for all $j \in [k]$, we could estimate the BCE decomposition.

Factoring the Posterior Distribution

Because

- CNNs have local dependence of outputs on the inputs
- The AWGN channel is *memoryless*, i.e. noise is added to each output value independently

we can factor

$$\mathbb{P}[\mathbf{u}_j = u|\mathbf{y}] \propto \sum_{\mathbf{u}\in\mathbb{F}_2^k, \mathbf{u}_j = u} \prod_{i=1}^{k} \mathbb{P}[\mathbf{y}_i|\mathbf{u}_{\texttt{wind}}]$$

where `wind` comprises the inputs that determine $\mathbf{y}_i$. Naively computing this expression will result in $\Omega(2^k)$ computations.

Idea: What if we group together common terms in the sum-product?

Junction Tree Algorithm

- Pick an ordering to "eliminate" bit positions $j \in [k]$ from the sum
- Use the elimination ordering to turn our factor graph into a *Junction Tree*

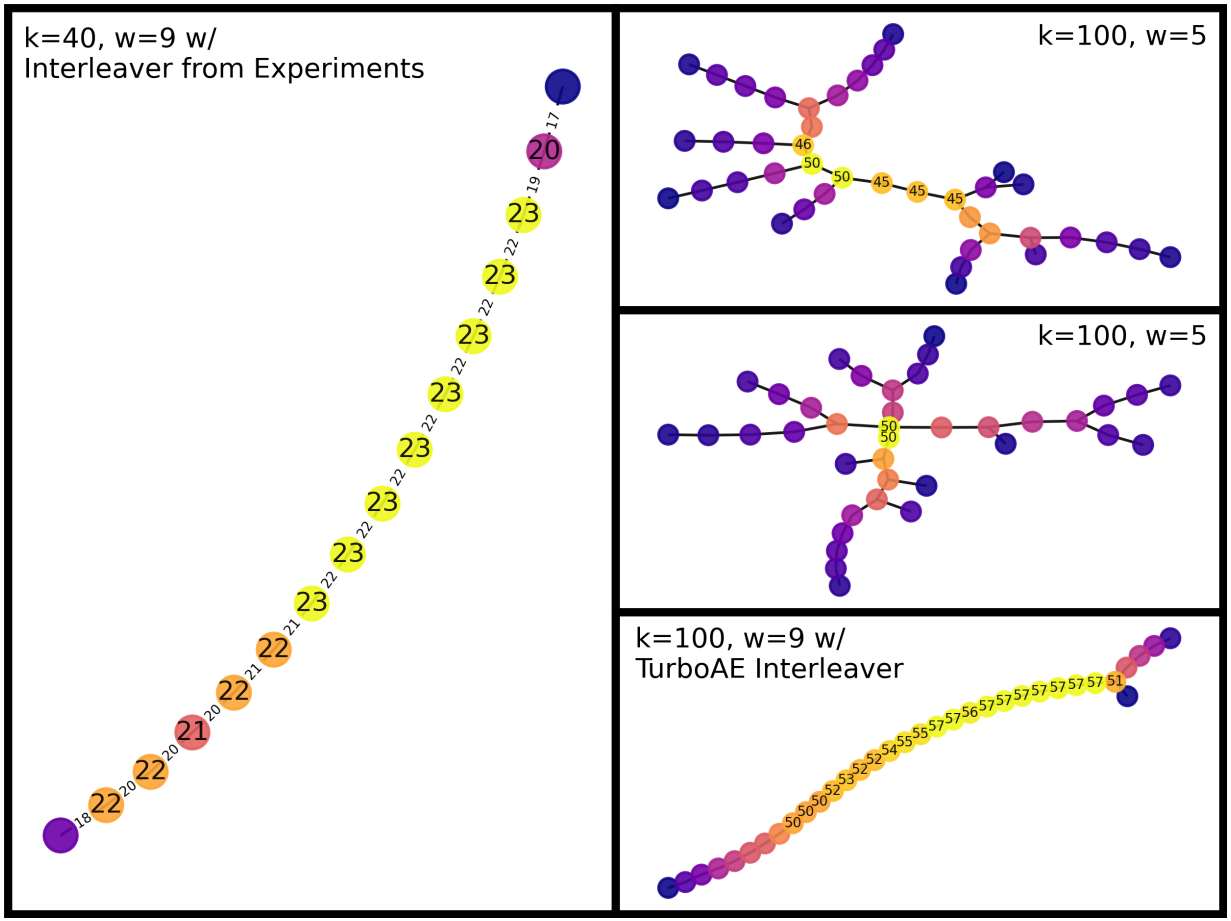
Definition

Let G be a factor graph with nodes $\mathcal{X}$ and factors Φ over $\mathcal{X}$. A **Junction Tree** (aka *Clique Tree*) T on G is a tree so

1. Each node C of T corresponds to a subset of $\mathcal{X}$ (called a cluster)
2. T is *family-preserving*: Every factor $\phi \in \Phi$ corresponds to a cluster C of T that contains that factor.
3. T has the *running-intersection property*: for every pair of clusters C_1, C_2 of T, variables $C_1 \cap C_2$ belong to all clusters on the unique path from C_1 to C_2.

Junction Tree Algorithm

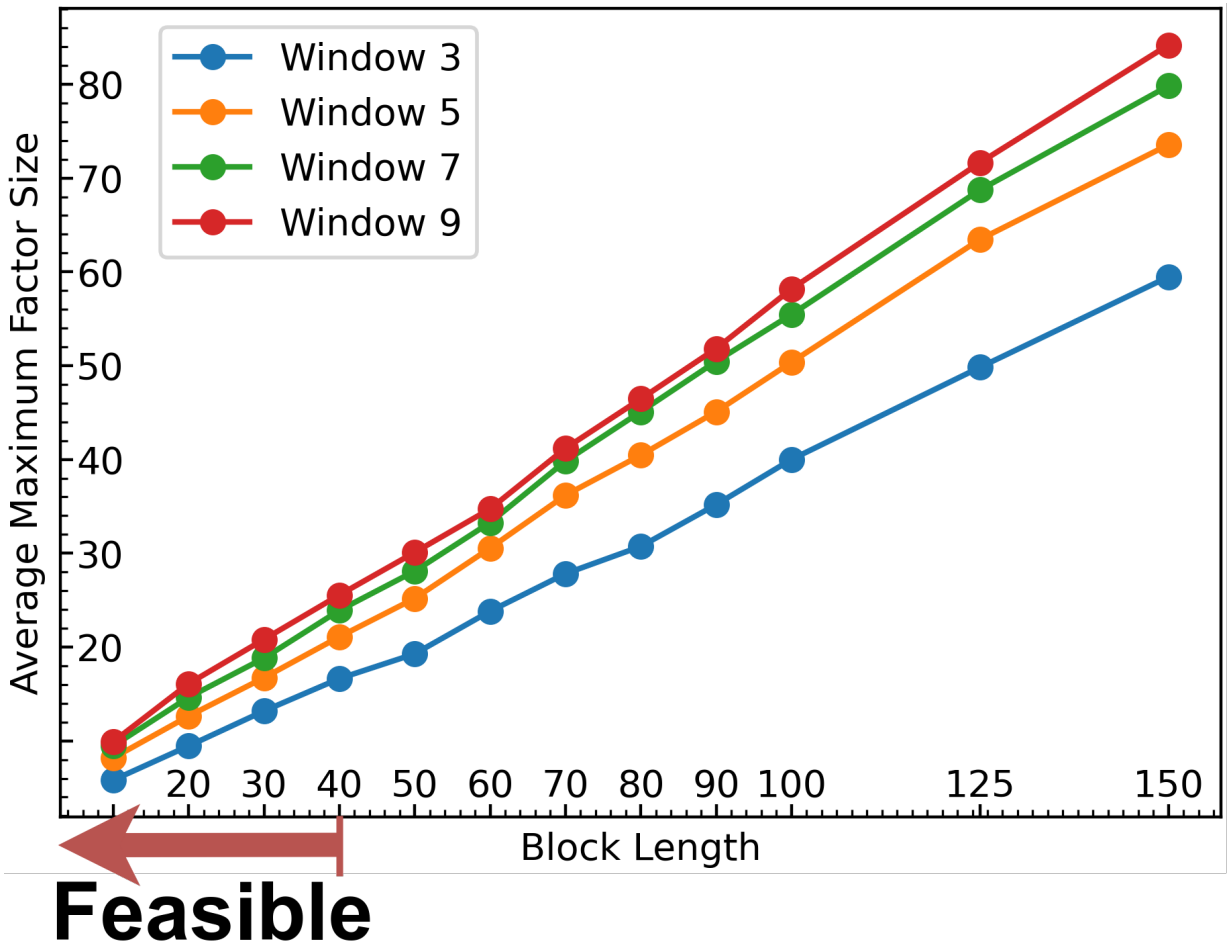
- We can apply the *Belief Propagation* algorithm to a Junction Tree to simultaneously compute the posteriors on all U_j.



- Node annotations and colors correspond to cluster sizes
- Runtime is exponential in cluster size

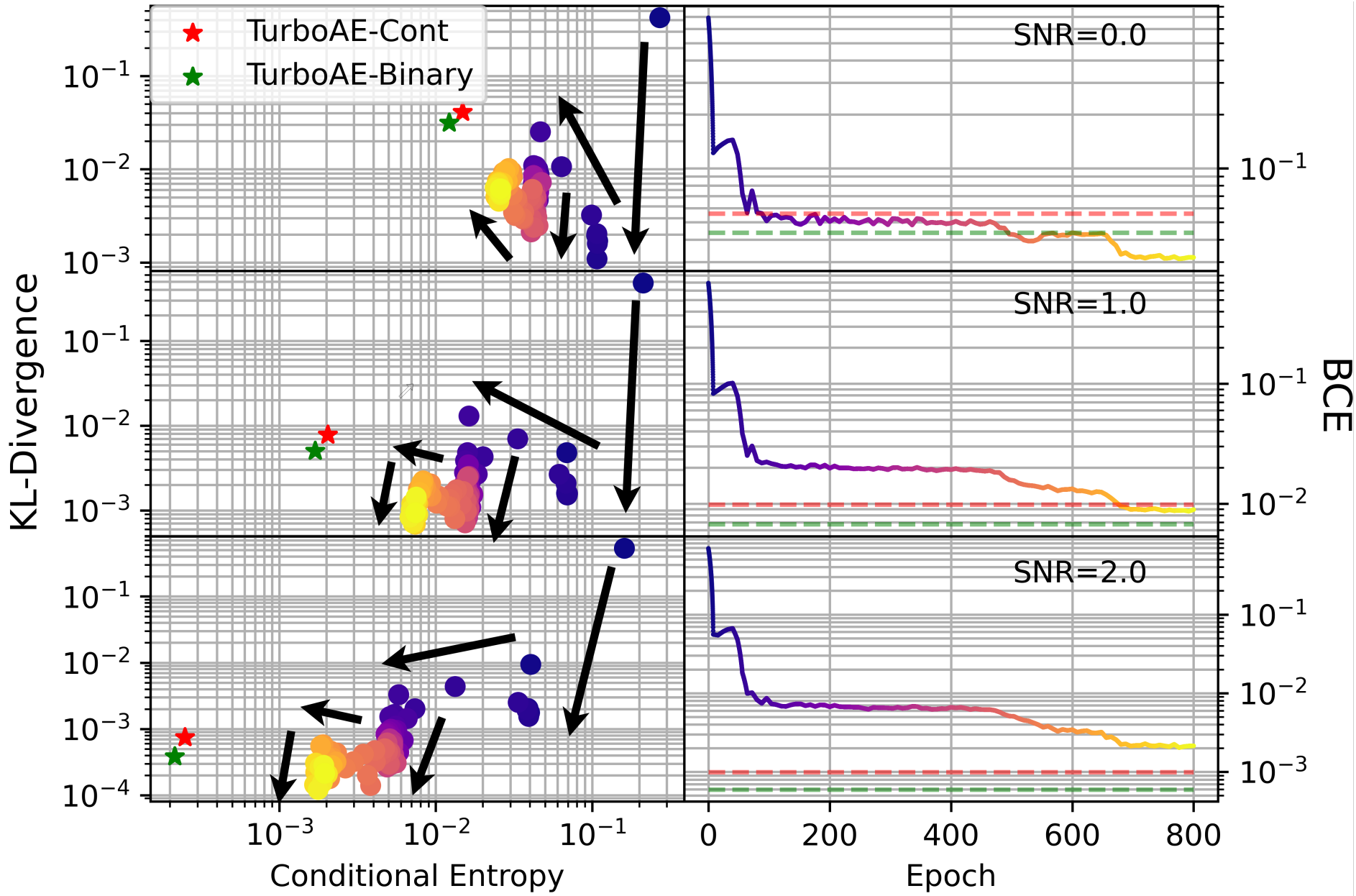
Junction Tree Algorithm

- Finding a minimal complexity Junction Tree is an NP-hard problem, but heuristics can work well.

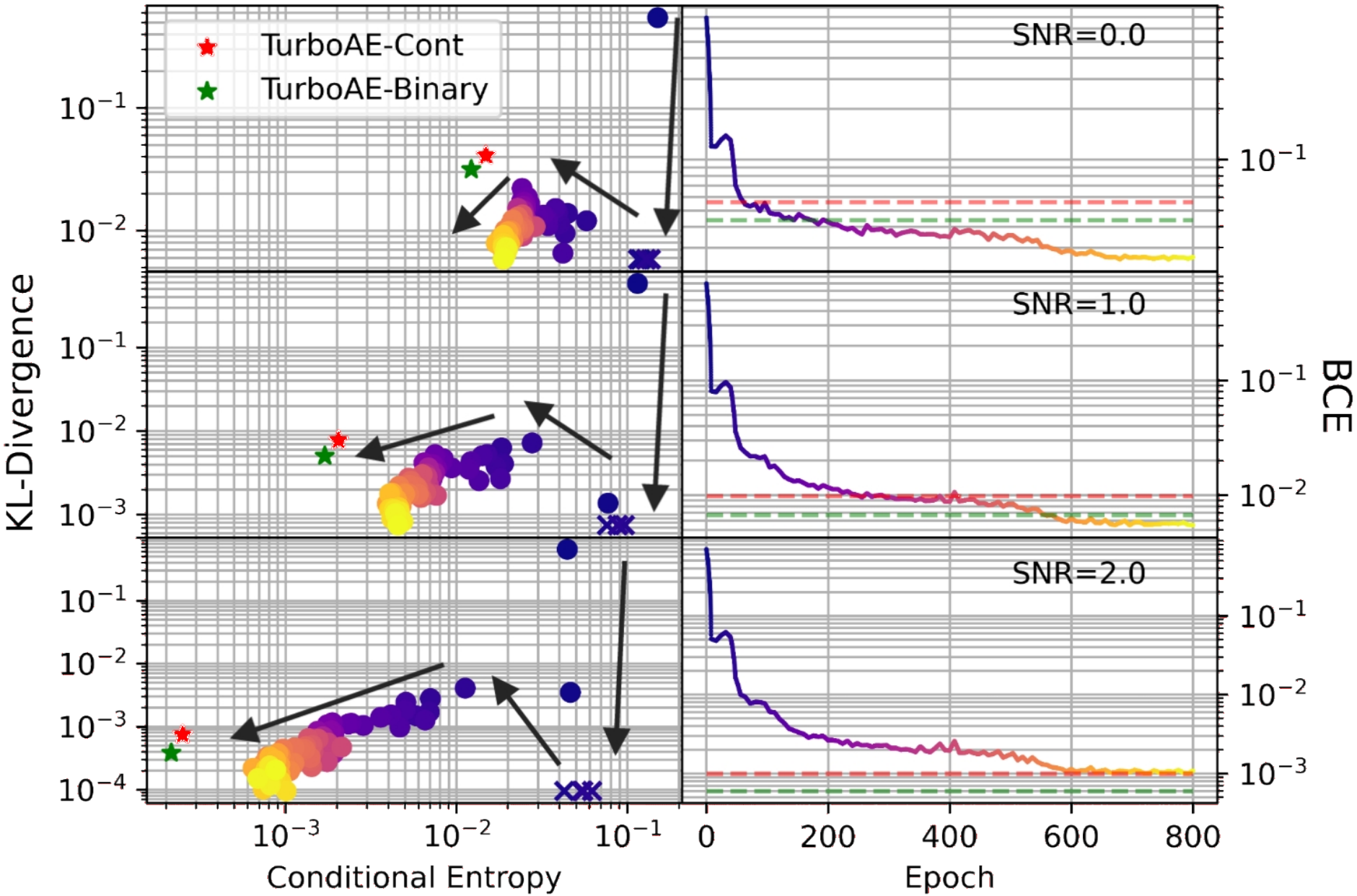


- Maximum cluster size scales roughly like half of k using heuristics

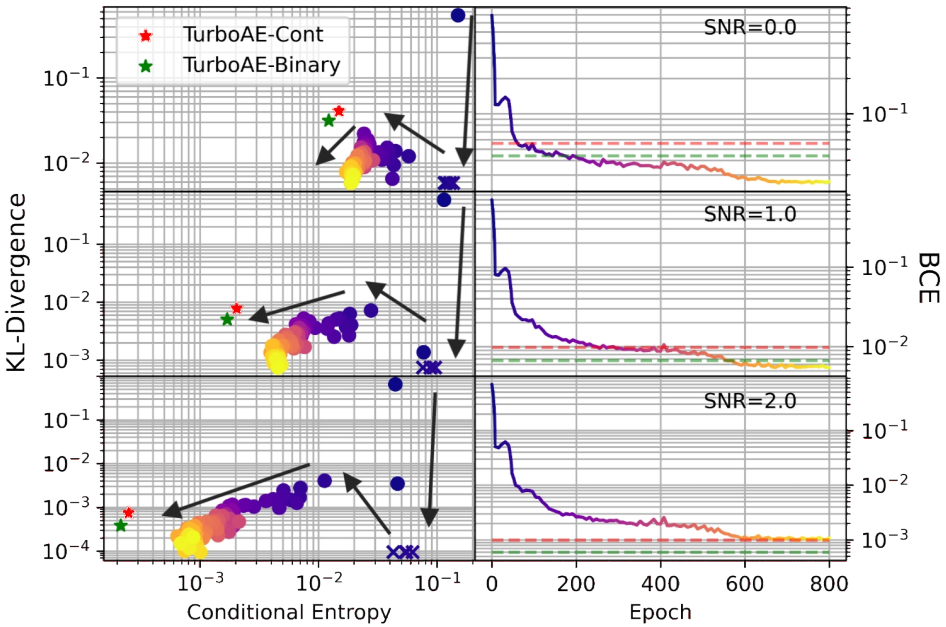
Decomposing BCE Training Evolution



Decomposing BCE Training Evolution



Decomposing BCE Training Evolution



- Training exhibits saddle-to-saddle dynamics
- Improvement out of saddle corresponds to improvement in encoder
- Zig-zag behavior is on scale of hundreds of epochs

Conclusions

We can understand the training process of TurboAE by

- Decomposing BCE into an encoder and decoder terms
- Estimating the terms using the Junction Tree algorithm during training

Findings

- Learning tends to happen in "spurts", with sudden improvements in encoder
- Decoder quickly catches up to changed encoder
- "Hardest" part of learning is optimizing the encoder

Further questions

- What properties of an encoder determine how hard it is to find a better nearby encoder?
- Do these training dynamics also occur when learning other DL-ECCs?

References I


Sebastian Cammerer, Fayçal Ait Aoudia, Sebastian Dörner, Maximilian Stark, Jakob Hoydis, and Stephan ten Brink, *Trainable Communication Systems: Concepts and Prototype*, IEEE Transactions on Communications **68** (2020), no. 9, 5489–5503.


N. Devroye, N. Mohammadi, A. Mulgund, H. Naik, R. Shekhar, Gy. Turán, Y. Wei, and M. Žefran, *Interpreting Deep-Learned Error-Correcting Codes*, ISIT, June 2022, pp. 2457–2462.


N. Devroye, A. Mulgund, R. Shekhar, Gy. Turán, M. Žefran, and Y. Zhou, *Interpreting Training Aspects of Deep-Learned Error-Correcting Codes – extended ArXiv version*, June 2023.


Yihan Jiang, Hyeji Kim, Himanshu Asnani, Sreeram Kannan, Sewoong Oh, and Pramod Viswanath, *Turbo autoencoder: Deep learning based channel codes for point-to-point communication channels*, NIPS, December 2019, pp. 2758–2768.

References II


Hyeji Kim, Yihan Jiang, Sreeram Kannan, Sewoong Oh, and Pramod Viswanath, *Deepcode: Feedback codes via deep learning*, IEEE Journal on Sel. Areas in Inf. Theory **1** (2020), no. 1, 194–206.


Ashok Vardhan Makkuva, Xiyang Liu, Mohammad Vahid Jamali, Hessam Mahdavifar, Sewoong Oh, and Pramod Viswanath, *Ko codes: inventing nonlinear encoding and decoding for reliable wireless communication via deep-learning*, ICML (2021).


A. Mulgund, R. Shekhar, N. Devroye, Gy. Turán, and M. Žefran, *Evaluating interpretations of deep-learned error-correcting codes*, 2022 58th Annual Allerton Conference on Communication, Control, and Computing (Allerton), September 2022, pp. 1–8.

References III


Emre Ozfatura, Yulin Shao, Alberto Perotti, Branislav Popović, and Deniz Gündüz, *All you need is feedback: Communication with block attention feedback codes*, IEEE Jour. on Sel. Areas in Info. Theory (2022), 1–1.

Variable Elimination (VE)

We can more carefully compute this expression by factoring. For example, $k = 5$

$$\begin{aligned}
\mathbb{P}[U_1 = 1|Y] &\propto \sum_{U_{2:5} \in \mathbb{F}_2^4} \mathbb{P}[Y_1|U_{1,3}]\mathbb{P}[Y_2|U_{2,4}]\mathbb{P}[Y_3|U_{1,5}]\mathbb{P}[Y_4|U_{3,4}]\mathbb{P}[Y_5|U_{2,3}] \\
&= \sum_{U_{2:4}} \mathbb{P}[Y_1|U_{1,3}]\mathbb{P}[Y_2|U_{2,4}]\mathbb{P}[Y_4|U_{3,4}]\mathbb{P}[Y_5|U_{2,3}] \sum_{U_5} \mathbb{P}[Y_3|U_{1,5}] \\
&= \tau^{(1)} \sum_{U_{2:3}} \mathbb{P}[Y_1|U_{1,3}]\mathbb{P}[Y_5|U_{2,3}] \sum_{U_4} \mathbb{P}[Y_2|U_{2,4}]\mathbb{P}[Y_4|U_{3,4}] \\
&= \tau^{(1)} \sum_{U_3} \mathbb{P}[Y_1|U_{1,3}] \sum_{U_2} \mathbb{P}[Y_5|U_{2,3}]\tau_{2,3}^{(2)} \\
&= \tau^{(1)} \sum_{U_3} \mathbb{P}[Y_1|U_{1,3}]\tau_3^{(3)}
\end{aligned}$$

Naive: 79 operations **Variable Elimination**: 23 operations